Лабораторная работа №10.
Применение алгоритмических конструкций при создании сценариев формирования Web-страниц с помощью PHP

Цель работы: изучить основы синтаксиса скриптового языка PHP, операторов ***if-else, switch, for***, ***while***, применить полученные знания для написания сценариев на PHP.

Теоретический материал

PHP (Hypertext Preprocessor – препроцессор гипертекста; первоначально, Personal Home Page Tools – инструменты для создания персональных веб-страниц») – скриптовый язык общего назначения, интенсивно применяемый для разработки веб-приложений. В настоящее время поддерживается подавляющим большинством хостинг-провайдеров и является одним из лидеров среди языков, применяющихся для создания динамических веб-сайтов.

PHP-сценарии могут размещаться в отдельном файле (с расширением *.php*) или встраиваются непосредственно в HTML документ.

Существует несколько способов внедрения кода PHP в HTML документы:

- С помощью открывающего тега ***<?php*** и закрывающего тега ***?>***.
- С помощью коротких тегов ***<?*** и ***?>***. Данная возможность доступна только при специальной настройке.
- С помощью тэгов *<script language="php">* и *</script>*
- Путем использования echo тэгов в стиле ASP: ***<%*** и ***%>***. Такая возможность доступна при соответствующей конфигурационной настройке.

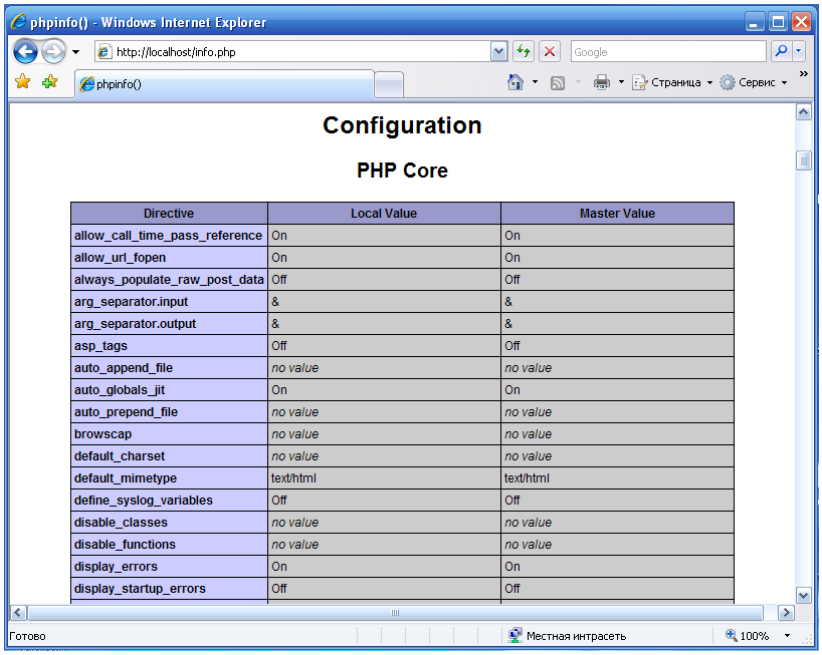
В дальнейшем в примерах будет использоваться первый из вариантов внедрения PHP кода.

Код, который находится внутри указанных тэгов, обрабатывается интерпретатором PHP, весь остальной код остается неизменным.

Для того, чтобы увидеть текущие настройки PHP, и для проверки его работоспособности полезно использовать специальную функцию *phpinfo()*:

```
<?php
phpinfo();
?>
```

После выполнения этого кода, в веб-браузере можно будет увидеть примерно следующее (показана небольшая часть):

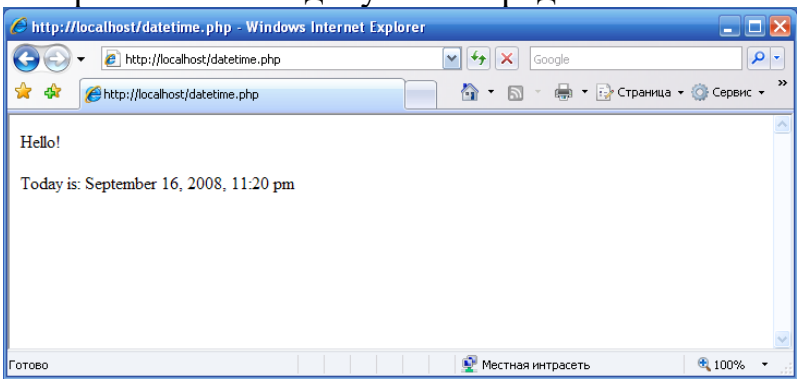


В целом же листинг содержит информацию об установленных опциях и расширениях PHP, версии PHP, информацию о веб-сервере и переменных окружения, информацию о версии ОС, путях, настройках конфигурационных переменных, полях заголовка HTTP и PHP лицензии.

Следующий пример демонстрирует вариант с внедрением PHP кода в HTML:

```
<html>
<body>
<p>Hello! </p>
<p>Today is:
<?php
 $today = date("F j, Y, g:i a");
echo($today);
?>
</p>
</body>
</html>
```

Результат обработки этого документа представлен ниже:



Одной из главных задач, решаемых с помощью PHP, является обработка данных, получаемых от пользователя через веб-формы. Рассмотрим, каким образом в PHP реализуется такая обработка.

```
<?php
$method = $_SERVER["REQUEST_METHOD"];
if ($method == "GET") $query = "_GET";
elseif ($method == "PUT") $query = "_PUT";
else die("$method is not supported!");
print "<p><b>Method</b>: $method </p>";
print "<p><u>Params:</u></p>";
foreach ($$query as $name => $value){
print "<b>$name</b> = <i>$value</i> <br>";
}
?>
```

В первой строке сценария с помощью переменной окружения REQUEST_METHOD из глобального ассоциативного массива $_SERVER определяется метод передачи данных в запросе клиента. В зависимости от выбранного метода переданные данные будут извлекаться либо из глобальной переменной $_GET либо из $_PUT. Если метод отличается от GET или PUT, либо не определен, то происходит принудительное завершение работы сценария с выдачей сообщения через вызов функции die(). В принципе, можно также использовать глобальный массив $_REQUEST, содержащий внутри себя массивы $_GET, $_ POST и $_COOKIE, позволяющий избавиться от проверки метода передачи.

Конструкция вида $$query демонстрирует косвенное обращение к переменной, т.е. переменная $query содержит идентификатор другой переменной, и для обращения к ней необходимо добавить еще один знак $.

Обе переменные $_GET и $_ POST являются ассоциативными массивами, поэтому для перебора элементов был использован специальный оператор **foreach** (имя_массива **as** ключ => значение).

Следует обратить внимание на то, что внутрь строковых констант, ограниченных символами “ ” можно вставлять переменные. После обработки такой строки интерпретатором вместо переменной вставляется ее фактическое значение. Также для конкатенации строк можно использовать оператор ‘**.**’

При составлении программ часто необходимо выполнение различных действий в зависимости от результатов проверки некоторых условий. Для организации ветвлений используются операторы ***if / else*** и оператор ***switch***. Основной синтаксис оператора ***if/else*** следующий:

```
if(условие){
  // код, который выполняется, когда справедливо условие if
} else {
  // код, который выполняется, когда условие if ложно
}
```

Необходимо отметить открывающую и закрывающую скобки, { и }, в приведенном выше коде. Открывающая скобка сообщает коду, где начинается блок кода, а закрывающая скобка указывает коду, где блок заканчивается. Поэтому все между { и } выполняется как часть оператора ***if***.

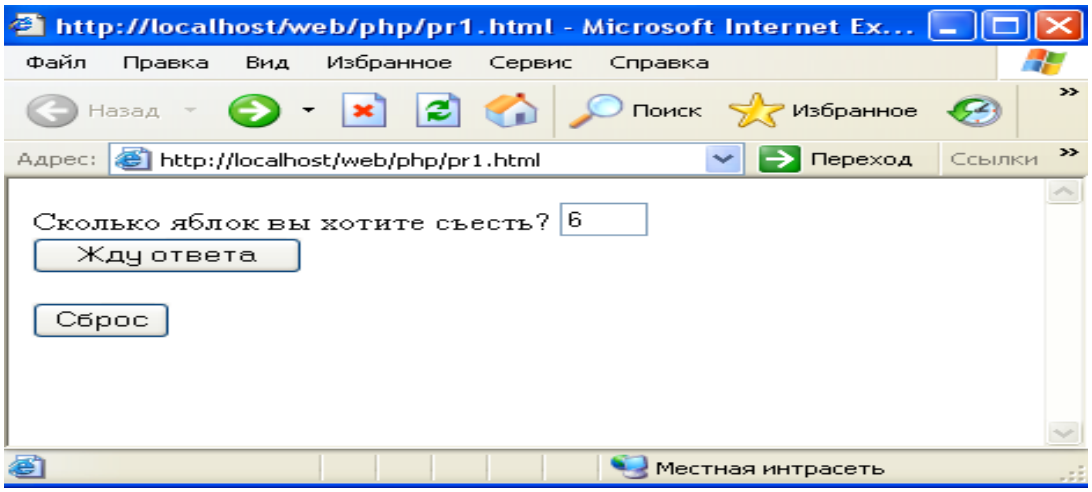
Необходимо отметить, что закрывающая скобка оператора ***if*** размещается непосредственно перед ключевым словом "***else***". Оператор ***else*** имеет свой собственный набор скобок и свой собственный блок для выполнения.

Пример 1. Требуется написать сценарий, который запрашивает у пользователя кол-во яблок, которое пользователь желает съесть (имеющееся кол-во яблок - 5). Если пользователь вводит число больше 5-ти, то выводится сообщение о том, что имеется, лишь 5 яблок, поэтому нельзя съесть большее кол-во. Если пользователь вводит число меньше 5-ти, то выводится сообщение о количестве оставшихся яблок.

Приведем реализацию задачи на HTML и PHP для рассматриваемого примера. С помощью HTML создается Web-форма (pr1.html) для запроса кол-ва яблок, которое пользователь желает съесть:

```
<html>
<body>
<form name="form1" action=example1.php method=POST>
Сколько яблок вы хотите съесть?
<input type="text" size =2 name="eat">
<br>
<input type="submit" value="Жду ответа">
<br><br>
<input type="reset">
</form>
</body>
</html>
```

При открытии страницы с кодом в браузере появляется окно с Web-формой:

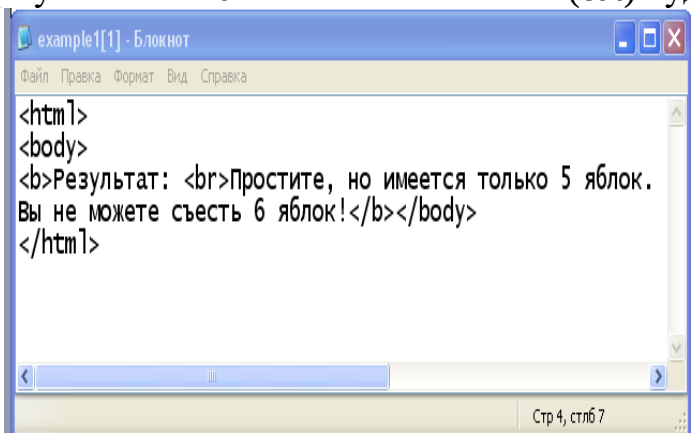


С помощью скрипта с PHP – кодом (example1.php) реализуется обработчик для вывода одного из требуемых сообщений:

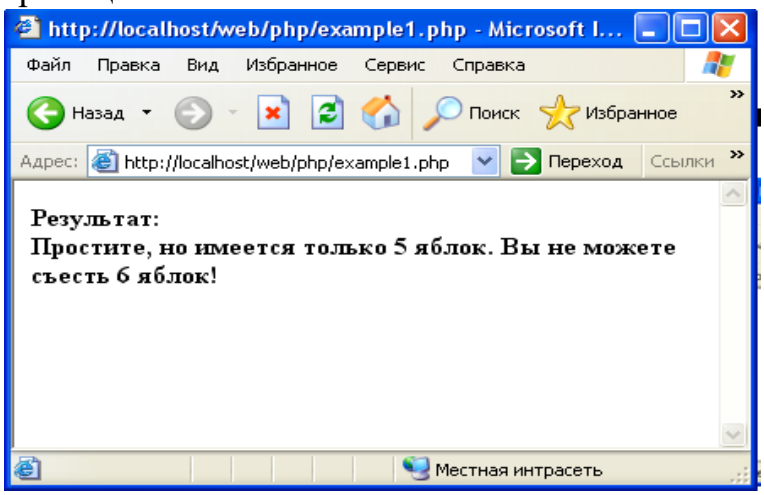
```
<html>
<body>
<?PHP
$eaten=$_POST['eat'];
$apples = 5;
if($eaten > 5){
```

```
  $s='Простите, но имеется только 5 яблок.  Вы не можете съесть
  '.$eaten.' яблок!';
} else {
  $apples -= $eaten;
  $s='А теперь имеется только '.$apples.' яблок!';
}
echo '<b>'.'Результат: <br>'.$s.'</b>';
?>
</body>
</html>
```

HTML – код, сгенерированный интерпретатором PHP на Web-сервере после ввода в форму значения 6 в поле кол-во яблок (eat) будет иметь вид:



При выполнении скрипта с помощью браузера (путем указания полного пути к нему в адресной строке) результат будет выглядеть как обычная Web-страница:



В тех случаях, когда при решении задачи требуется выбрать один вариант действия из нескольких возможных, удобно воспользоваться оператором **switch**:

```
switch(B){
case L1:S1;
case L2:S2;
.................
case Ln:Sn;
```

default:
}

где B-выражение; L1, L2,…, Ln – литералы; S1, S2, …, Sn – операторы.

Выполнение переключателя происходит так: вычисляется значение выражения B. Если значение B равно L1, выполняется оператор S1, а затем все остальные операторы, либо до первого оператора break, либо до конца оператора ***switch***. Если значение B равно L2, выполняется оператор S2, а затем все остальные операторы, либо до первого оператора break, либо до конца оператора switch и т.д. Если же значение B не равно ни одному из значений L1, L2, …,Ln выполняются операторы S. Часть ***default:S*** может отсутствовать, тогда оператор имеет вид:

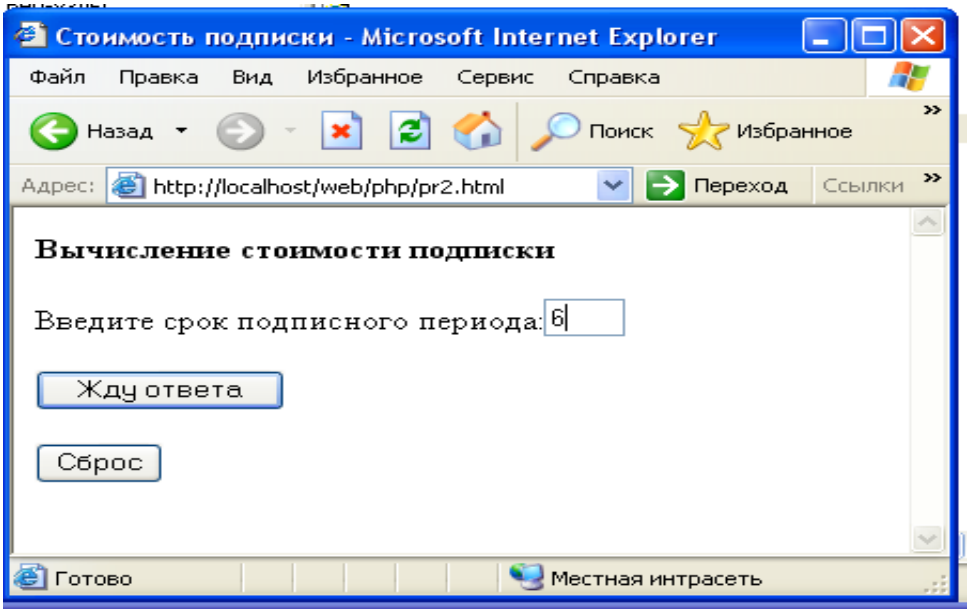
```
switch(B)
{case L1:S1;
case L2:S2;
.................
case Ln:Sn;
}
```

Пример 2. Предположим, что с помощью анкеты, размещенной на Web-странице, пользователь хочет подписаться на электронное издание. Стоимость редакционной подписки зависит от срока подписного периода. Если выбирается 1 месяц, то стоимость будет 60 рублей, 3 месяца – 168 рублей, 6 месяцев -312 рублей,12 месяцев – 540 рублей. Создадим документ, в котором пользователь вводит срок подписного периода (1, 3, 6, 12 месяцев), в зависимости от которого определяется стоимость подписки. В данном сценарии будем использовать оператор switch.

Приведем реализацию задачи на HTML и PHP для рассматриваемого примера. С помощью HTML создается Web-форма (pr2.html) для запроса срока подписного периода:

```
<html>
<title>Стоимость подписки</title>
<body>
<form name="form1" action=example2.php method=POST>
<h4>Вычисление стоимости подписки</h4>
Введите срок подписного периода:<input type="text" size =2
name="num"><br>
<br>
<input type="submit" value="Жду ответа">
<br><br>
<input type="reset">
</form>
</body>
</html>
```

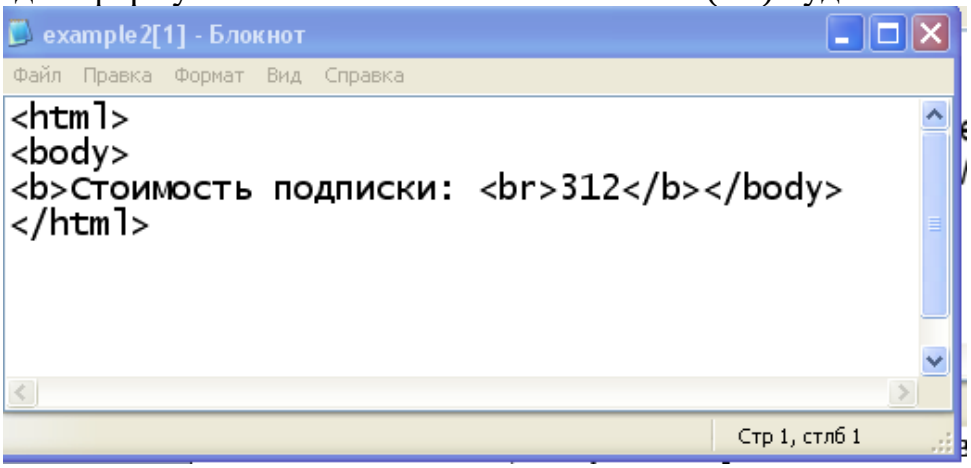
При открытии страницы с HTML-кодом в браузере появляется следующее окно с Web-формой:



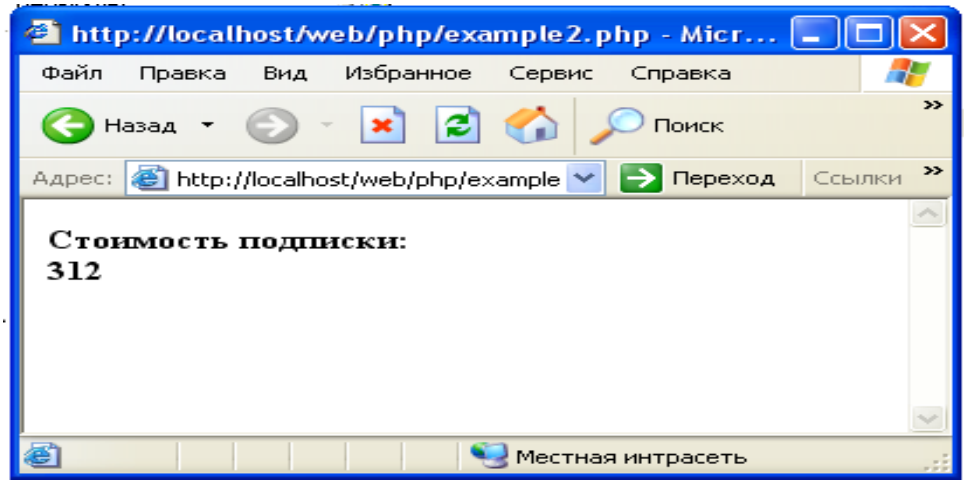
С помощью скрипта с PHP – кодом (example2.php) реализуется обработчик для подписки и расчета стоимости:

```
<html>
<body>
<?PHP
$n=$_POST['num'];
switch($n){
case 1: $s=60; break;
case 3: $s=168; break;
case 6: $s=312; break;
case 12 :$s=540; break;
default: $s="неверен срок подписного периода";
}
echo '<b>'.'Стоимость подписки: <br>'.$s.'</b>';
?>
</body>
</html>
```

HTML – код, сгенерированный интерпретатором PHP на Web-сервере после ввода в форму значения 6 в поле кол-во яблок (eat) будет иметь вид:



Результат, представленный в браузере:



Оператор for создает цикл с тремя необязательными выражениями, заключенными в круглых скобках и разделенными точкой с запятой, сопровождаемыми блоками утверждений, выполненных внутри цикле:

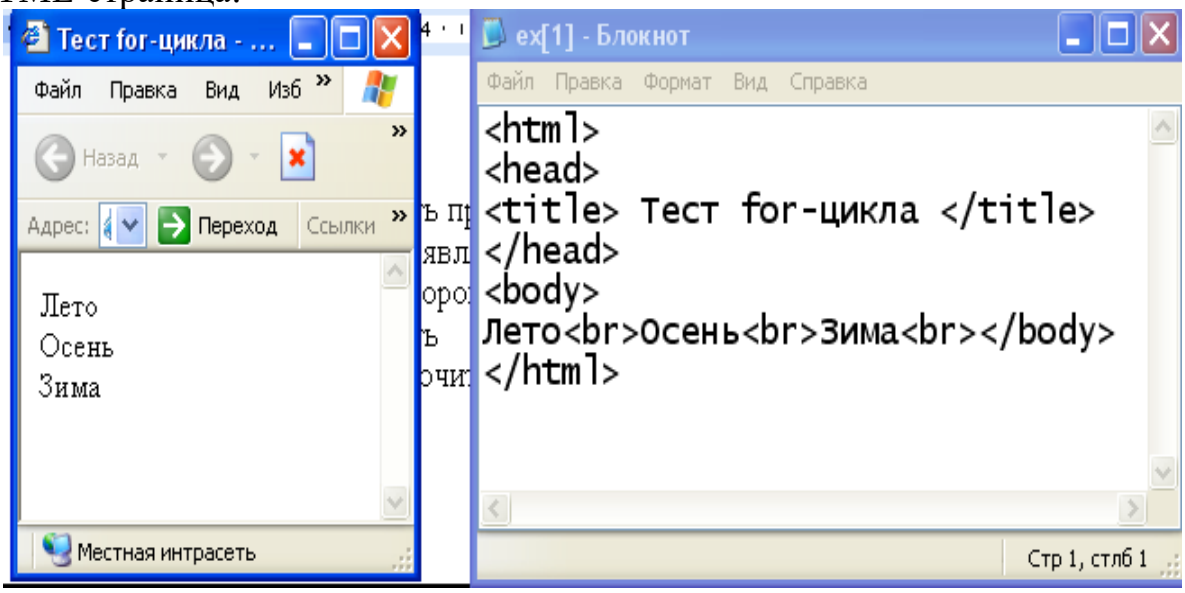
```
for (initialExpression;condition;updateExpression) {
  ...строки кода...
}
```

где выражение ***initialExpression*** используется для инициализации переменной счетчика, которую можно создать с помощью оператора *var*. (это выражение необязательно), выражение ***condition***, которое вычисляется на каждом проходе через цикл (если это условие истинно, то выполняются условия внутри цикла; это условие необязательно; если оно опущено, то условие всегда считается равным истинному, тогда цикл продолжается до ошибки или до оператора ***break)***, выражение ***updateExpression*** используется для изменения значения переменной счетчика (оно тоже является необязательным; в этом случае можно обновлять значение переменной счетчика внутри цикла).

Приведем **пример** скрипта на языке PHP для вывода содержимого массива с помощью for-цикла.

```
<html>
<head>
<title> Тест for-цикла </title>
</head>
<body>
<?php
$fotos[]="Лето";
$fotos[]="Осень";
$fotos[]="Зима";
for ($n=0; $n<count($fotos); $n++) {
echo "$fotos[$n]"."<br>";
}
?>
</body>
</html>
```

В результате выполнения этого скрипта будет получена следующая HTML-страница:



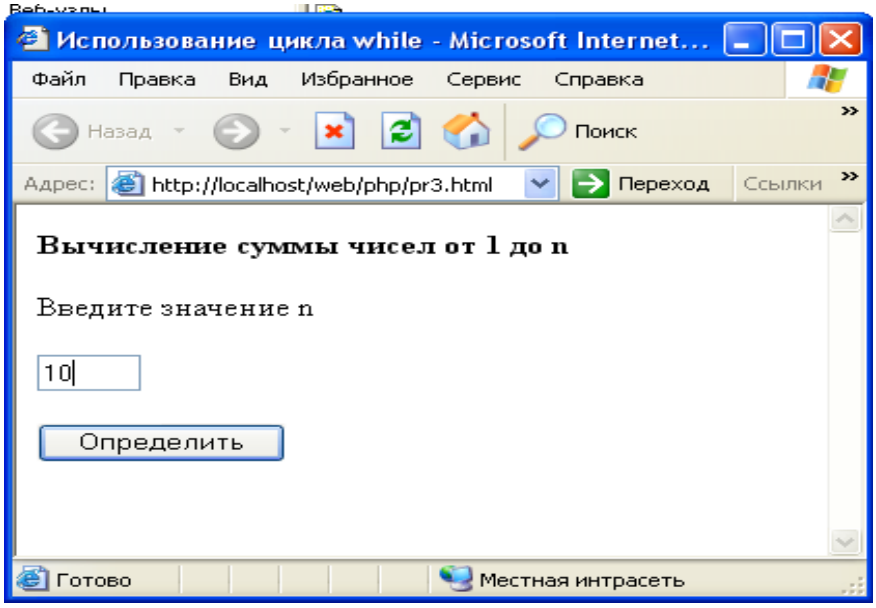
Оператор ***while*** – это оператор цикла, который повторяет цикл, пока значение *condition* есть истинно (true). Как только значение ***condition*** становится ложным (***false***), то управление переходит к первому оператору после фигурной скобки, закрывающей тело цикла ***while***:

```
while (condition) {
  ...statment...
}
```

Пример 3. Создать Web-страницу с формой для вычисления суммы чисел от 1 до n.

Для создания Web-формы напишем следующий HTML – код (pr3.html):

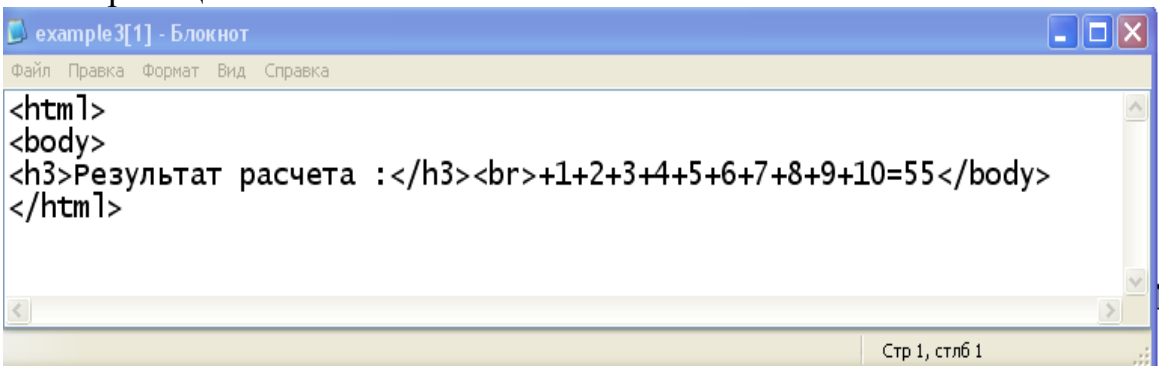
```
<html>
<head>
  <title> Использование цикла while </title>
</head>
  <h4>Вычисление суммы чисел от 1 до n</h4>
  <p> Введите значение n</p>
  <form name ="form1"action=example3.php method=POST>
  <input type="text" name="n" size =4> <br><br>
  <input type="submit" value="Определить"
 </form>
</body>
</html>
```

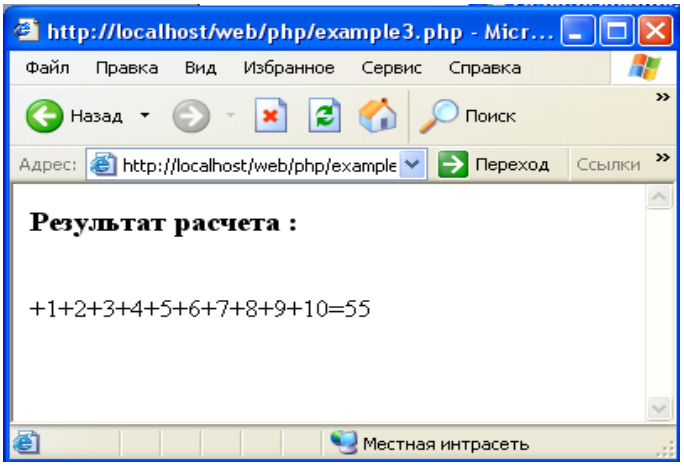


Вычисление суммы чисел будет производиться в PHP-скрипте (example3.php), который запускается после отправки Web-формы на обработку (кнопка Определить):

```
<html>
<body>
<?PHP
 $n=$_POST['n'];
 $i=0;
 $result=0;
 echo '<h3>'.'Результат расчета :'.'</h3><br>';
 while($i<$n) {
 if ($i%20 == 0 && $i>0) echo '<br>';
 $result+= ++$i;
 echo "+".$i;
 }
 echo '='.$result;
 ?>
</body>
</html>
```

В результате выполнения этого скрипта будет получена следующая HTML-страница:





Оператор ***do . . . while*** – это оператор цикла с пост условием, который повторяет цикл, пока значение ***condition*** есть истинно (***true***). Как только значение *condition* становится ложным (false), то управление переходит к первому оператору после ключевого слова ***while***:

```
do {
  ...statment...
}
while (condition)
```

Оператор ***break*** прерывает текущий цикл ***for*** и ***while***, передавая управление первому оператору после цикла.

Например поиск номера стоп-элемента в массиве может быть реализован в следующем цикле:

```
$i = 0;
while ($i < 10) {
   if ($arr[$i] == "stop") { break; }
   $i++;
}
```

Оператор continue передает управление оператору проверки истинности условия в цикле ***while*** и оператору обновления значения счетчика в цикле ***for***. Важное отличие от оператора ***break*** заключается в том, что оператор **continue** не прерывает цикл, а делает следующее:

- в цикле ***while*** переходит назад к условию;
- в цикле ***for***, это переходит к модернизированному выражению.

В следующем примере в цикле реализована обработка значений с нечетными индексами некоторого массива:

```
while (list($key, $value) = each($arr)) {
  if ($key % 2) { // skip even members
      continue;
  }
 do_something_odd($value);
}
```

В приведенном примере **list()** используется для присвоения значений списку переменных (**$key** и **$value**), функция **each()** используется для

извлечения ключа и значения текущего элемента массива **$arr** и перемещения внутреннего указателя массива вперед. **do_something_odd() –** это вызов функции, которая получает в качестве параметра значение элемента массива **$value** с нечетным номером и выполняет с ним некоторые действия.

В PHP имеется широкий диапазон функция для работы с файлами:

- **fopen**(путь_к_файлу, тип_доступа) – открытие файла;
- **feof**(указатель_на_файл) – проверка на наличие признака конца файла;
- **fgets**(указатель_на_файл) – чтение строки из файла;
- **fclose**(указатель_на_файл) – закрытие файла.

Для записи данных в файл можно использовать функцию **fputs**(указатель_на_файл, строка).

Например, следующий пример демонстрирует чтение файла, в котором находится выполняемый PHP код:

```
<?php
$fh = fopen("read.php","r");
if (!$fh) die("Cannot open file");
while (!feof ($fh)) {
$line = fgets($fh);
echo $line,"<br>";
}
fclose($fh);
?>
```

В следующем примере сгенерированные функцией rand() псевдослучайные числа сохраняются в файле rand.dat:

```
<?php
$n = 10;
$fh = fopen("rand.dat","w");
if (!$fh) die("Cannot open file");
srand();
for ($i=0; $i < $n; $i++){
$d = rand(0,100);
fputs($fh,"$d\n");
}
fclose($fh);
?>
```

Пример 4. В качестве итогового примера можно рассмотреть страницу и сценарий, реализующий четыре арифметические операции над целыми числами.

Код Web-страницы с формой-калькулятором:

```
<html>
<body>
<form action='http://localhost/Scripts/test.pl'>
<p>Operand1: <input type='text' name='A'></p>
```

```
<p>Operand2: <input type='text' name='B'></p>
<p>Operation:<br>
<select name='op'>
<option value='+'>+</option>
<option value='-'>-</option>
<option value='*'>*</option>
<option value='/'>/</option>
<select></p>
<input type='submit' value='Calculate!'>
</from>
</body>
</html>
```

Код PHP-сценария для обработки формы-калькулятора:

```
<?php
$method = $_SERVER["REQUEST_METHOD"];
if ($method == "GET") $query = "_GET";
elseif ($method == "PUT") $query = "_PUT";
else die("$method is not supported!");
$q = $$query;
$a = $q["A"];
$b = $q["B"];
$op = $q["op"];
switch ($op){
case '+': $result = (int)$a + (int)$b; break;
case '-': $result = (int)$a - (int)$b; break;
case '*': $result = (int)$a * (int)$b; break;
case '/': {
if ($b == '0') die("divide by zero!");
}
default: die("operator $op is not defined");
}
print "<p>Result: $a $op $b = $result </p>";
print "<p><a
href=".$_SERVER['HTTP_REFERER']."">Back</a></p>";
?>
```

Общая постановка задачи

1. Выполнить два индивидуальных задания по варианту.
2. Поместить разработанный PHP-скрипт на FTP-сервере.
3. Составить электронный отчет о проделанной работе.

Список индивидуальных данных

Задание №1

1. Написать программу на HTML и PHP для следующего задания: Определить, каким является треугольник, заданный координатами своих вершин: равносторонним.

2. Написать программу на HTML и PHP для следующего задания: Упорядочить 3 числа по убыванию.

3. Написать программу на HTML и PHP для следующего задания: Найти максимальную цифру в записи данного трехзначного числа.

4. Написать программу на HTML и PHP для следующего задания: Определить, находится ли точка с координатами (x, y) в I или III четверти и внутри круга данного радиуса r с центром в начале координат.

5. Написать программу на HTML и PHP для следующего задания: Даны координаты вершин треугольника: (ax, ay), (bx, by), (cx, cy). Определить, лежит ли данный треугольник внутри окружности радиуса R с центром в точке (x0, y0).

6. Написать программу на HTML и PHP для следующего задания: Даны действительные числа x, y, z. Выяснить, существует ли треугольник с длинами сторон x, y, z.

7. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 3 вещественных чисел. Определить количество введенных четных чисел.

8. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 3 вещественных чисел. Определить сумму введенных нечетных чисел.

9. Написать программу на HTML и PHP для следующего задания: Даны действительные числа a, b, c, d. Если они упорядочены по невозрастанию, то каждое число заменить наибольшим из них; если они упорядочены по неубыванию, то числа оставить без изменения; в противном случае все числа заменить их квадратами.

10. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 3 вещественных чисел. Определить количество введенных нулей.

11. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 5 вещественных чисел. Определить число, предшествующее первому из введенных минимальных значений.

12. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 3 вещественных чисел. Определить количество максимальных значений.

13. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 3 вещественных чисел. Определить максимальное число.

14. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 6 вещественных чисел. Вычислите сумму тех из них, которые нечетны и отрицательны.

15. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 6 вещественных чисел. Определить сколько раз в последовательности меняется знак при переходе к следующему элементу.

16. Создать документ с формой, содержащей 2 поля для ввода и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид. В первое поле пользователь должен вводить целое число, во второе поле выводится строка «четное число», если введенное число является четным и «нечетное» в обратном случае.

17. Создать документ с формой, содержащей 2 поля для ввода и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид. В первое поле пользователь должен вводить число, во второе поле выводится квадратный корень из числа, в случае если введенное число отрицательно, то выводится сообщение «действительного корня не существует».

18. Создать документ с формой, содержащей 2 поля для ввода и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид. В первое поле пользователь должен вводить число, во второе поле выводится модуль введенного числа.

19. Создать документ с формой, содержащей 2 поля для ввода и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид. В первое поле пользователь должен вводить число от 1 до 12 , во второе поле выводится название месяца, с порядковым номером равным введенному числу, в случае ввода несуществующего номера месяца выводится сообщение - «введите число от 1 до 12».

20. Создать документ с формой, содержащей 2 поля для ввода и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид. В первое поле пользователь должен вводить название дня недели, во второе поле выводится с порядковый номер введенного дня, в случае ввода несуществующего дня недели выводится сообщение - «нет такого дня».

21. Создать документ с формой, позволяющей вводить коэффициенты квадратного уравнения и выводить корни уравнения, либо сообщение о том, что действительных корней не существует.

22. Создать документ с формой, позволяющей задать уравнение параболы, и определить пересекает ли она ось X или нет.

23. Создать документ с формой, содержащей 2 текстовых поля и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид. В первое поле пользователь должен вводить название города (задать в программе 5 городов), во второе поле выводится температура воздуха в городе, если данных о температуре введенного города нет, то выводится сообщение «таких данных нет».

24. Создать документ с формой, содержащей 1 список, 2 текстовых поля и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид, для оформления заказа покупки в Интернет магазине. В магазине продаются ручки по 6 руб., карандаши – по 3 руб. и фломастеры – по 15 руб. Пользователю предоставляется возможность выбрать из списка

товаров какой-то один и ввести количество товара, которое он хочет купить. После составления заказа, в поле формы выводится стоимость заказа.

25. Создать документ с формой, содержащей 1 список,1 текстовую область и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид, для вывода информации о члене вашей группы. Пользователь должен выбрать в списке фамилию члена вашей группы, а в текстовой области формы выводится информация о дате рождения этого человека (или какая-то другая информация).

Задание №2

1. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 10 вещественных чисел. Вычислите сумму тех из них, которые нечетны и отрицательны.
2. Написать программу на HTML и PHP для следующего задания: Найти максимальную цифру в записи заданного числа.
3. Написать программу на HTML и PHP для следующего задания: Вывести запись числа n в обратном порядке.
4. Написать программу на HTML и PHP для следующего задания: Проверить является ли число n степенью числа 3.
5. Написать программу на HTML и PHP для следующего задания: Переставить первую и последнюю цифры числа n.
6. Написать программу на HTML и PHP для следующего задания: Определить сколько четных цифр в числе n.
7. Написать программу на HTML и PHP для следующего задания: Определить чему равна сумма цифр числа n.
8. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 10 вещественных чисел. Найдите количество положительных, отрицательных и нулевых чисел.
9. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 10 вещественных чисел. Найдите среднее арифметическое положительных чисел.
10. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 10 вещественных чисел. Вычислите разность между максимальным и минимальным числами.
11. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 10 вещественных чисел. Вычислите сумму тех из них, которые кратны 5.
12. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 10 вещественных чисел. Определите: содержит ли последовательность хотя бы одну пару равных соседних элементов.
13. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 10 вещественных чисел. Определите: сколько раз меняется знак при переходе к следующему элементу.
14. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 10 вещественных чисел. Определите: является ли последовательность чисел возрастающей.

15. Написать программу на HTML и PHP для следующего задания: Определите все простые трехзначные числа.

16. Написать программу на HTML и PHP для следующего задания: На экране в форме вводятся 10 вещественных чисел. Вычислите разность между максимальными и минимальными числами.

17. Написать программу на HTML и PHP, позволяющую найти факториал числа.

18. Написать программу на HTML и PHP, позволяющую пользователю вводить число и степень, в которую необходимо возвести это число. В форму выводится результат возведения в степень (использовать циклы).

19. Написать программу на HTML и PHP, позволяющую найти количество цифр, во введенном пользователем числе.

20. Написать программу на HTML и PHP, позволяющую найти сумму цифр, во введенном пользователем числе.

21. Написать программу на HTML и PHP, которая позволяет узнать является ли число, введенное пользователем простым.

22. Написать программу на HTML и PHP, которая позволяет узнать сумму целых чисел в некотором диапазоне. Диапазон вводится пользователем.

23. Написать программу на HTML и PHP, которая позволяет узнать количество четных чисел в некотором диапазоне. Диапазон вводится пользователем.

24. Написать программу на HTML и PHP, которая позволяет узнать количество чисел, которые делятся на 7 в некотором диапазоне. Диапазон вводится пользователем.

25. Написать программу на HTML и PHP, позволяющую пользователю задать кол-во повторений для фразы «Привет». После задания числа повторений вывести в тело документа фразу «Привет» указанное число раз.